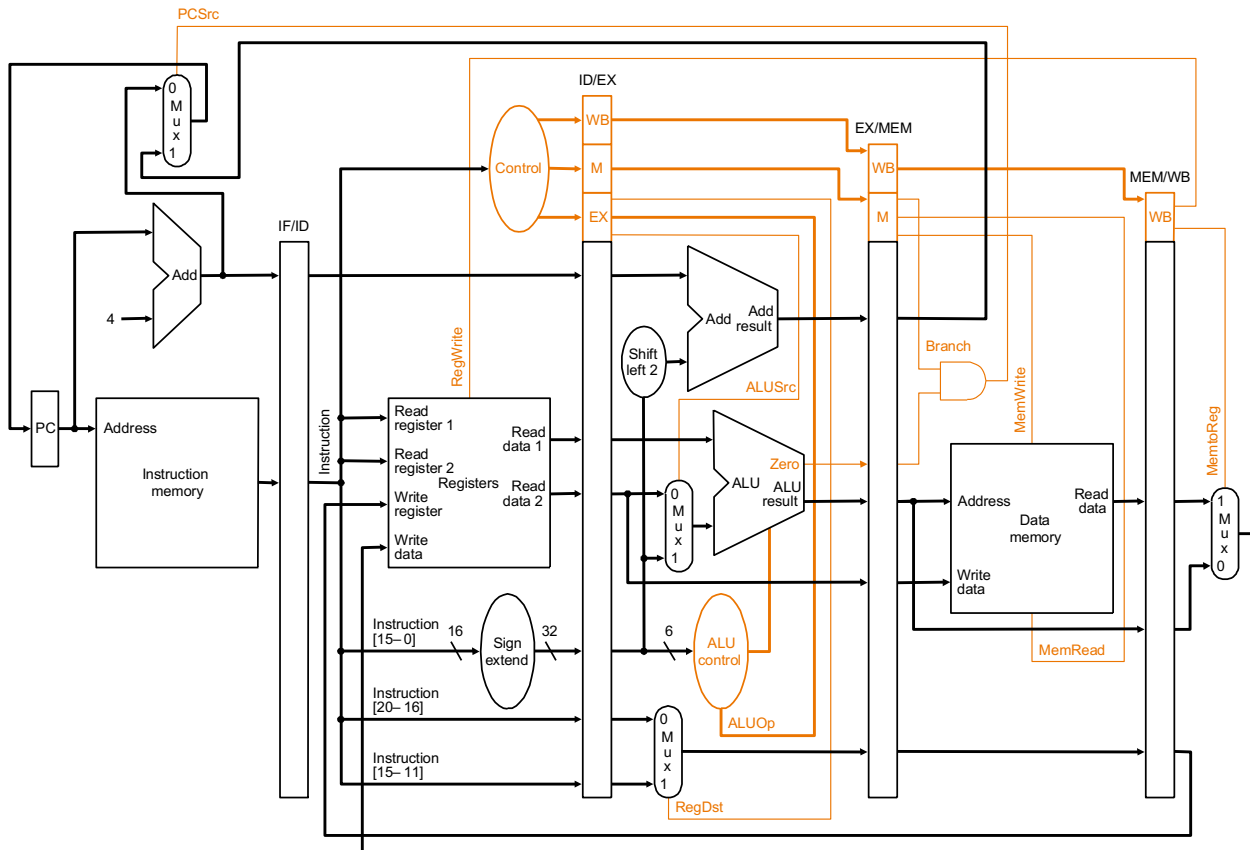


Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (10/01)

1. Consider the simple **five-stage pipeline** from the textbook (COD Figure 4.51) shown below.



In this pipeline, control signals for each instruction are set when the instruction is in the **Decode (ID)** stage. Specify which of the following control signals need to be stored in the **EX/MEM pipeline register**. Recall that a control signal is stored in a pipeline register if it may be used by any instruction at any subsequent stages.

- (a) RegWrite
- (b) RegDst
- (c) ALUSrc
- (d) MemWrite
- (e) MemRead
- (f) MemtoReg

Answer:

(a), (d), (e) and (f)

2. Consider the following MIPS assembly code, running on the 5-stage pipeline above.

```

L0:  addi  $s1, $zero, 6
L1:  lw    $s3, 0($s4)
L2:  ori   $s2, $zero, 7
L3:  sw    $s0, 4($s4)
L4:  add   $s2, $s1, $s1

```

The following **pipeline diagram** shows which stage each instruction is performing at each cycle: F=Fetch (IF), D=Decode (ID), X=Execute (EX), M=Memory (MEM) or W=Writeback (WB).

instruction	clock cycle								
	1	2	3	4	5	6	7	8	9
L0	F	D	X	M	W				
L1		F	D	X	M	W			
L2			F	D	X	M	W		
L3				F	D	X	M	W	
L4					F	D	X	M	W

- (a) Specify all cycles when **ID/EX.ALUSrc** (i.e., the ALUSrc control bit stored in the ID/EX pipeline register) is set to 1.

Answer:

Cycles 3, 4, 5 and 6

- (b) Specify all cycles when **EX/MEM.RegWrite** (i.e., the RegWrite control bit stored in the EX/MEM pipeline register) is set to 1.

Answer:

Cycles 4, 5, 6 and 8

- (c) Assume that in the beginning of cycle 4, a fault occurs that forces **ID/EX.RegDst** (i.e., the RegDst control bit stored in the ID/EX pipeline register) to get stuck at 0 for the rest of the program. What are the contents of registers \$s1 and \$s2 at the end of the program due to this fault?

Answer:

**This stuck-at bug means all instructions in X that writes to registers will write to rt instead of rd. L0 writes 6 to \$s1 as there's no fault until L1 hits X. L2 writes 7 to \$s2, because 'ori' already writes to rt. L4 is changed to \$s1=\$s1+\$s1.
[s1] = 12 & [s2] = 7**

3. Assume you are performing the following **branch** instruction on the **five-stage pipeline** implementation above (COD Figure 4.51):

beq \$s0, \$s0, LOOP

In this implementation, at which **stage** of the branch instruction is the **PC** being set to the branch target address (i.e., LOOP)?

Answer:

MEM